AI 서비스 Mini-project 과제 보고서 — Base AI Agent의 RunBook 선택 개선

79번 박건우 (G079 · 광주 3반)

- 과제 선택: **1번 — Base AI Agent 기능 개선/추가**
- 대상 코드: `all_total_vue_student` (FastAPI + LangGraph v2 + MCP 2서버 인시던트 대응 에이전트)
- 개선 방향: RAG 기반 RunBook 선택 · 데이터 확장 · 평가 하네스 구축
- Git 주소: https://github.com/pigpgw/skala-ai-service-mini
- 보고서 원문: `doc/reports/result_report.md` (PDF 동봉) · 발표자료: `doc/reports/presentation.pdf`

---

1. 진행 사항 소개

1.1 무엇을 골랐나

기존 Base AI Agent는 장애 질의를 받으면 MCP로 모니터링 데이터를 조회하고, 장애 유형을 정해 대응 런북(runbook)을 골라 답변을 만든다. 이 구조는 그대로 두고, **장애 유형을 고르는 부분**과 **그걸 측정하는 부분**을 손봤다.

제가 잡은 개선 축은 네 가지다.

#	축	한 줄 요약
1	RAG 기반 RunBook 선택	키워드 첫 매칭 → 런북 임베딩 의미 검색
2	라우팅 기준 확대	예시 질의가 장애로 인식되지 않던 공백 수정
3	데이터 확장	모니터링 6→12시스템, 런북 6→10종(신규 유형 4종)
4	평가 하네스 구축	실행 1회 = 폴더 1개 하네스 + 테스트 65개 + 회귀 세트

1.2 무엇이 문제였는지 먼저 찾아봤다

장애 유형 판정이 `agent/graph.py::_classify_incident_type()` 한 함수에 압축돼 있었고, 키워드를 **순서대로** 보면서 **처음 맞는 것**을 고르고 있었다. 세 가지 문제가 있었다.

1. **규칙 순서가 증상 우선순위와 무관** — CPU가 timeout보다 앞에 있어, 타임아웃 질의가 `cpu-spike` 로 분류됐다.
1. **질의문을 아예 보지 않음** — "디스크 부족" 신고가 지연 알림에 가려 `high-latency` 가 됐다.

2. **맞는 키워드가 없으면 무조건 0** — "아무 장애나 db-down 대응을 내놔라"가 됐다.

고치기 전에 기준선을 먼저 재렸다. 질의 12개를 고정해서 돌렸더니 **8/12 = 0.6667**이었다 (`eval/baseline_keyword.json`). 오분류 4개 중 B07·B09는 규칙 순서, B10·B11은 질의문을 안 읽는 것이 원인이었다.

여기서 판단을 한 번 했다. 규칙을 늘리는 방법도 있겠지만, 새 장애 유형마다 사람이 우선순위를 다시 맞추는 방식은 계속 남는 숙제 같았다. 그래서 의미 검색으로 교체했다.

1.3 추가·수정한 task

task 1 — RAG 기반 RunBook 선택

- `agent/rag/embedder.py` — Ollama 임베딩(`qwen3-embedding:0.6b`, 1024차원). Ollama가 없으면 `HashingEmbedder`로 떨어지며, 이때는 품질 비교 근거로 쓰지 않는다. 실제로 어떤 임베더가 쓰였는지는 `/health`와 측정 manifest에 이름으로 남긴다.
- `agent/rag/index.py` — 런북 10종을 임베딩해 JSON 캐시로 저장. 무효화는 파일 목록 + mtime + size + 임베더 이름으로 판정하고, 어떤 인덱스로 측정했는지 `index_sha256`으로 사후에 특정할 수 있게 했다.
- `agent/rag/classifier.py` — 판정을 5분기로 나눴다. `disabled / index_unavailable / embed_failed / low_score / low_margin`이면 기존 keyword 경로로 내려간다. 즉 **RAG가 확신하지 못하면 예전 동작 그대로**다. 개선으로 기존 동작이 깨지지 않게 하는 안전장치로 두고 측정으로 확인했다.
- 연결은 `agent/graph.py::_select_incident_type()`가 담당한다. `replan_check_node`가 "정보 부족 → 장애유형 추가 조사"를 결정하는 지점이라 여기에 넣었고, 선택 근거 (유형·출처·유사도·1·2위 격차)는 `AgentState` 5개 키로 남겨 `/health`, `/chat` metadata, SSE `ReplanCheck` 이벤트로 노출했다.

task 2 — 라우팅 기준 확대

과제 PDF 9쪽이 알려준 회귀 질의 10개를 실제로 서버에 던져 봤다 (`eval/run_regression_queries.py`). **10개 중 3개만 통과**했다.

원인은 판정이 아니라 입구였다. 결정론 백엔드(`agent/fake_llm.py`)의 라우팅 정규식이 `장애|진단|다운|지연|급증|incident|티켓` 7개짜리였는데, PDF가 알려준 예시 질의 중 7개가 그 정규식에 걸리지 않았다. 걸리지 않으면 `chat`으로 떨어져 **장애 파이프라인이 아예 돌지 않았다**("안녕하세요! 무엇을 도와드릴까요?"만 답함).

즉 과제가 알려준 테스트 질의로 자기 자신을 테스트할 수 없는 상태였다. 실제 LLM 백엔드 (ollama/openai)는 "장애·성능 저하·지연·진단·모니터링·티켓"을 프롬프트로 분류하므로 이 문제가 없는데, 결정론 백엔드만 좁았다. 그래서 기준을 같은 운영 어휘로 넓히고, **모니터링 대상 시스템명이 있으면** incident로 가게 했다(목록에 없는 `billing-service`도 포함 — 8번 질의의 의도대로). 잡담과 세션 기억 질문은 그대로 chat이다.

task 3 — 데이터 확장

- 시스템 6종 → 12종(`data/monitoring_extended.json`), 런북 6종 → 10종. 신규 유형 4개는 queue-backlog, cert-expiry, connection-pool, replication-lag.

- 파일 선택은 `agent/data_profile.py` 한 곳에서만 한다(`DATA_PROFILE=base|extended`). 모니터링 경로가 `mcp_server.py` , `agent/graph.py` , LLM 모듈 2개, `eval/common.py` 총 다섯 곳에 흩어져 있었는데, 시스템을 늘릴 때 한 곳만 고치면 나머지가 6개 목록으로 잘려 "이름만 늘고 조사 대상은 그대로"인 가짜 확장이 된다.
- 기존 `data/monitoring.json` 은 덮어쓰지 않고 기준선으로 남겼다. 기준선 비교가 나중에도 가능해야 하기 때문이다.
- `billing-service` 는 일부러 넣지 않았다. 목록에 없는 시스템명을 물었을 때 unknown으로 답하는지 확인하는 회귀 질의(#8)의 대상이다.

task 4 — 평가 하네스 구축

- `eval/run_eval.py` — 실행 1회가 `artifacts/runs/<run_id>/` 폴더 1개가 된다. 폴더마다 실행 조건(백엔드·프로파일·임계값·데이터 해시·git 상태)을 담은 `manifest.json` , 지표를 담은 `metrics.json` , 입력 1건 = 1줄인 `requests.jsonl` 과 `responses.jsonl` 이 들어간다.
- `eval/metrics.py` — accuracy, macro-F1, 혼동 행렬, 카테고리별 정확도, 폴백률, 조사 커버리지, 지연 percentile, 재현성 판정을 한 파일에 모았다.
- `eval/labels_extended.json` — 40문항 라벨(10개 유형 × 4개 카테고리)을 **측정 전에 동결**했다. 라벨을 먼저 얼려 두고 나중에 손대면 "내가 만든 문제에 내가 맞힌 것"이 된다.
- `eval/run_regression_queries.py` — PDF 9쪽 회귀 질의 10개를 실제 서버에 던져 확인한다. 10번은 서버를 실제로 재시작해 세션 기억 유지까지 본다.
- `eval/check_reported_numbers.py` — **문서에 적힌 숫자가 측정 원본과 같은지** 기계로 대조한다.
- `tests/` — unit 6종 + integration 2종, 65개.

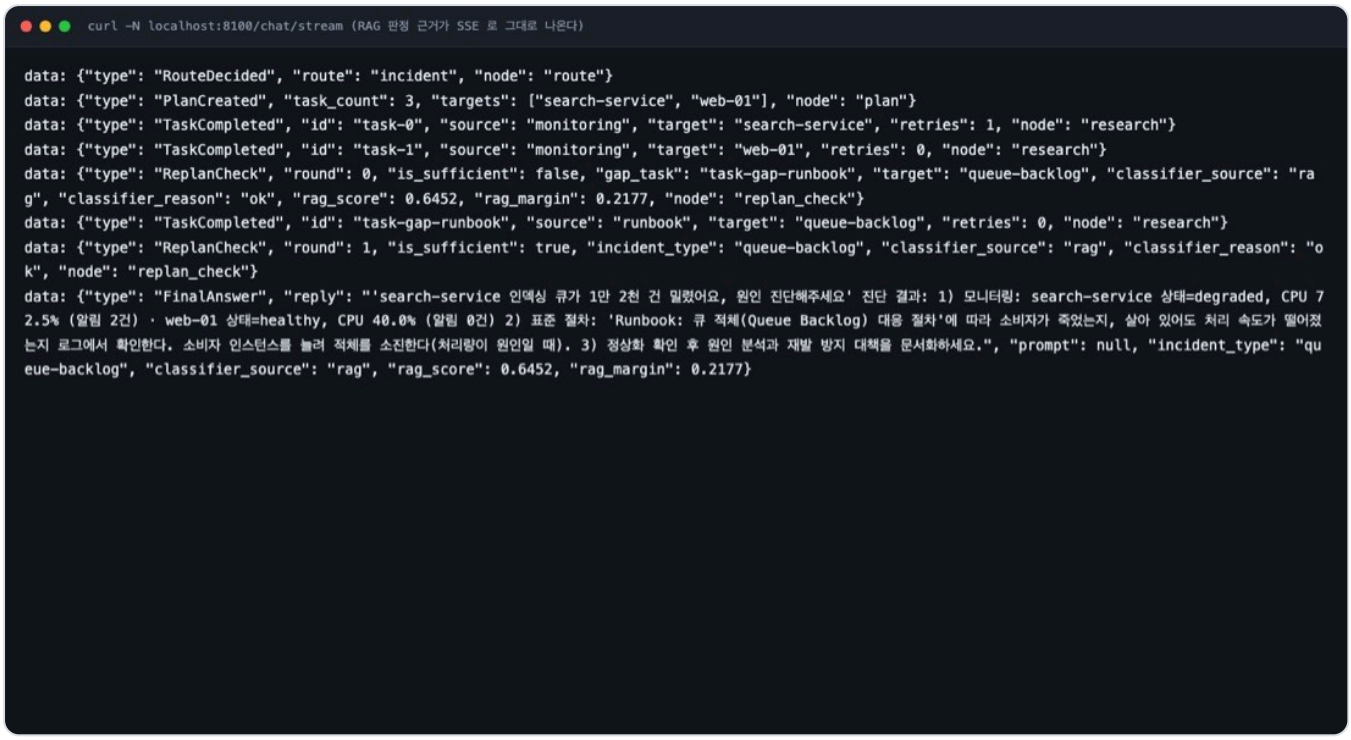
---

2. 사용한 기술 요소

기술 요소	어디에 구현돼 있는지	상태
Multi-Agent / LangGraph	`agent/graph.py` StateGraph (노드 10개), `agent/supervisor_v1.py` legacy	기존 코드 활용
MCP	`mcp_server.py` (ops 서버 — Tool·Resource·Prompt 실사용), `agent/mcp_client.py` (stdio 클라이언트). `mcp_server_docs.py` 는 decoy라 쓰지 않음	기존 + 조회 경로 확장
RAG	`agent/rag/` — `embedder.py` · `index.py` · `classifier.py`. `deps.rag` 로 주입되고 `graph.py::_select_incident_type()` 에서 호출	**신규 구현**
의존성 주입	`agent/deps.py` 의 `SupervisorDeps` 에 `rag` 필드 추가, `server.py` lifespan에서 번들 1회 생성 → 노드가 `os.environ` 을 직접 읽지 않는다	**확장**
LangSmith	`agent/graph.py` 노드에 `@traceable`, `agent/deps.py` · `server.py` 경유, `.env.example` 에 `LANGCHAIN_TRACING_V2` · `LANGCHAIN_PROJECT`	연결·구성 완료. 실 API 키 없는 환경이라 trace 캡처는 미수행
모듈화	`agent/` 하위 llm·graph·mcp·retry·rag·session_store 분리 + `doc/tools/` (문서 생성 도구) + `eval/` (평가 코드)	**확장**
단위 테스트	`tests/unit/` 6종(지표 계산기·데이터 프로파일·데이터 무결성·RAG 게이트·keyword 고정·라우터) + `tests/integration/` 2종(MCP stdio, FastAPI TestClient)	**신규 65개**

2.1 기능 캡처 — RAG 판정 근거가 SSE로 나온다

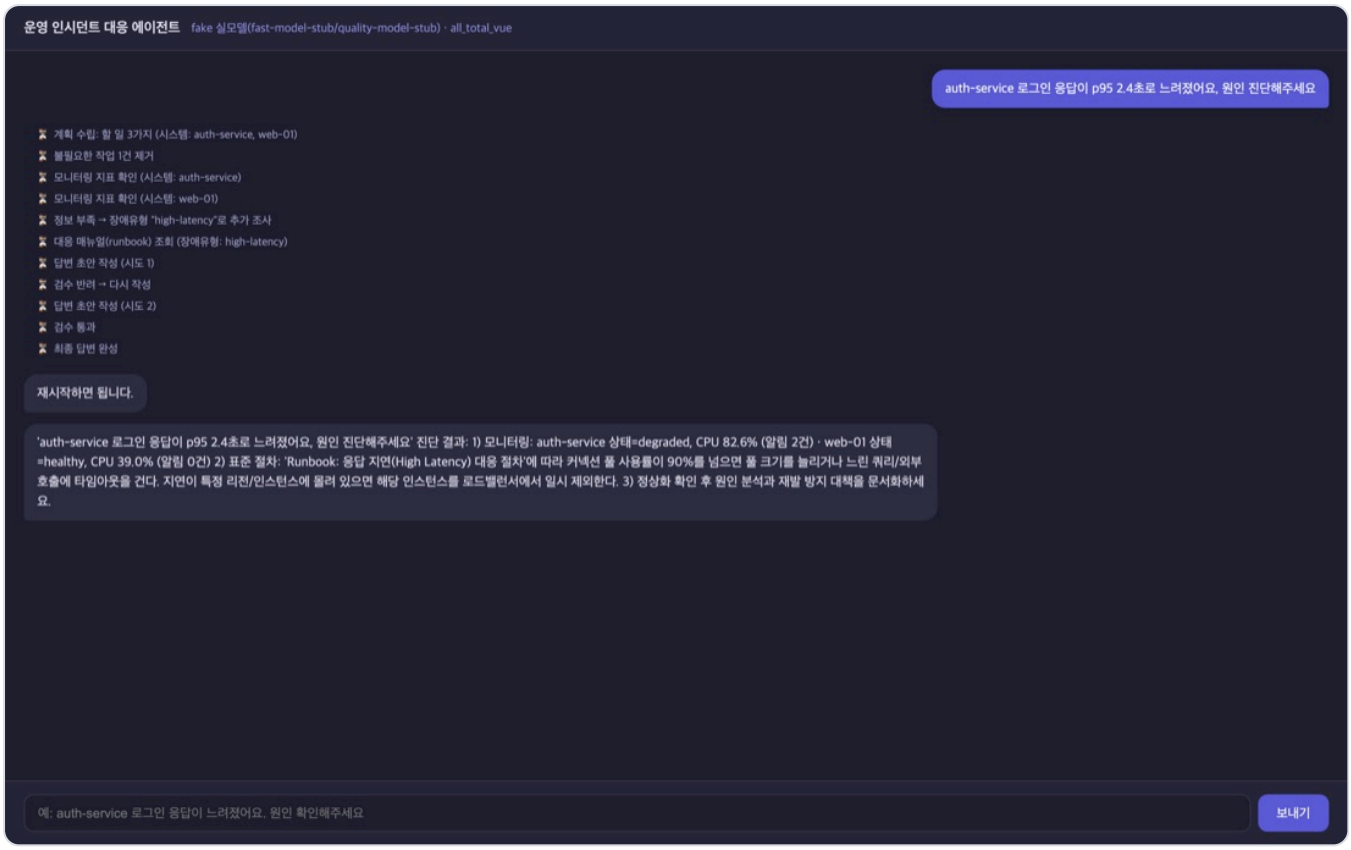
`replan_check_node` 는 "지금 모onitoring 정보만으로는 부족해 런북 조회가 필요하다"고 판단하고 런북을 추가로 조회한다. 그 판단의 근거(어떤 런북인지, 유사도 점수, 1·2위 격차)가 SSE로 그대로 나온다. 아래는 실제 `curl -N localhost:8100/chat/stream` 출력이다.



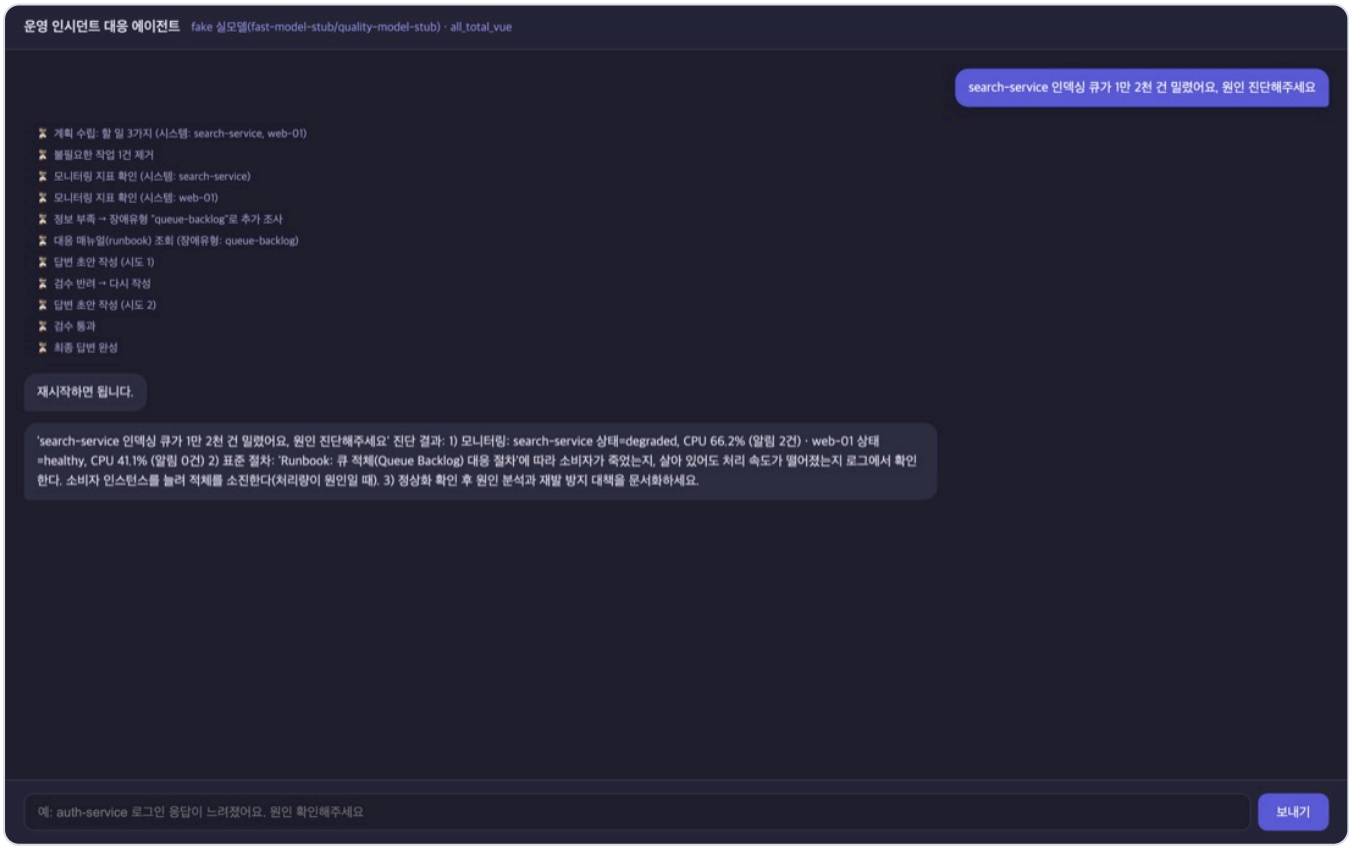
`"classifier_source": "rag"`, `"rag_score": 0.6452`, `"rag_margin": 0.2177`이 그대로 나온다. 같은 값이 `/chat` 응답의 `metadata`에도 들어가고, `/health`의 `rag` 블록에도 임베더·인덱스· 임계값이 함께 노출된다.

2.2 최종 실행 화면

브라우저 채팅 화면(`http://localhost:8600`)에서 "auth-service 로그인 응답이 p95 2.4초로 느려졌어요, 원인 진단해주세요"를 보낸 결과다. 조사 수집 → 정보 부족 → 장애유형 추가 조사 → 대응 매뉴얼 조회 → 초안 작성 → 검수 → 최종 답변이 순서대로 보인다.



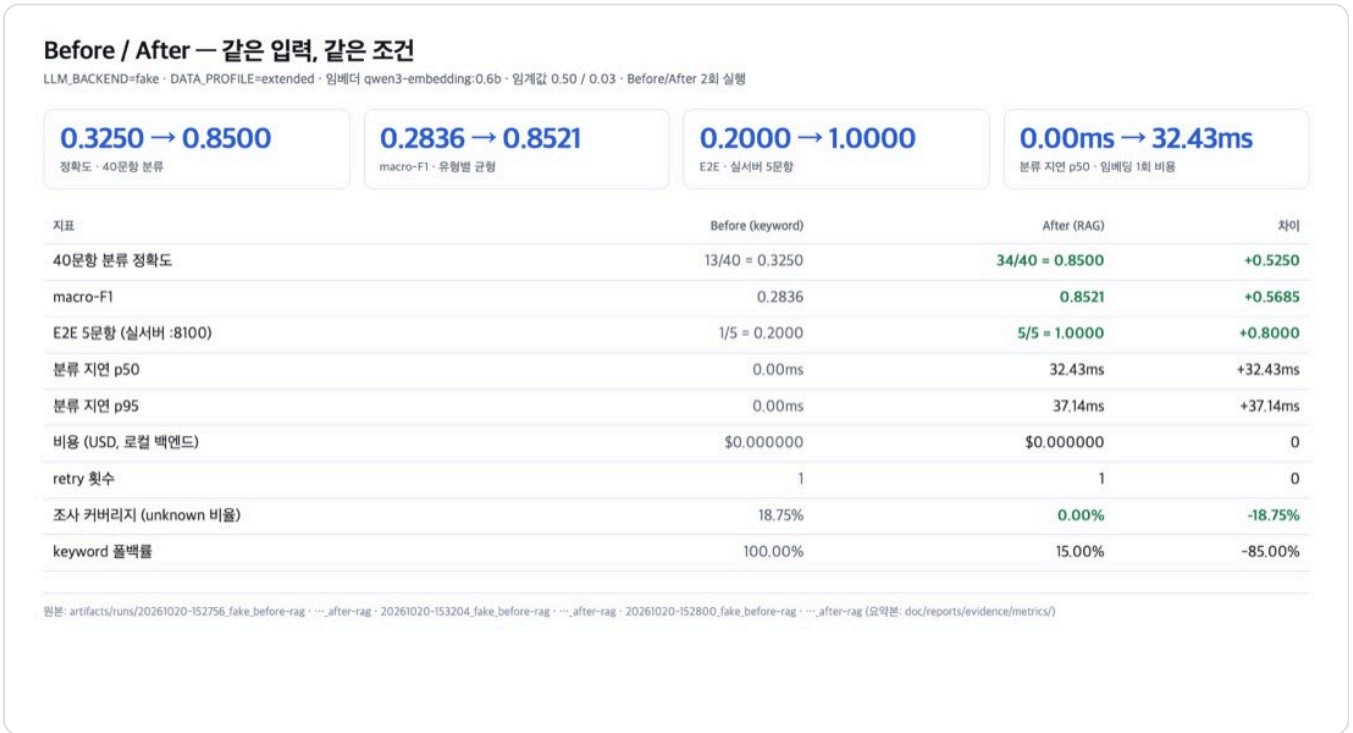
확장한 데이터로 처음 조사되는 시스템도 같은 화면으로 확인된다.



3. 선택 사항 — Before / After 실측 비교

같은 입력, 같은 백엔드, 같은 조건에서 2회 실행해 비교했다. 조건: `LLM_BACKEND=fake`, `DATA_PROFILE=extended`, 임베더 `ollama:qwen3-embedding:0.6b`, 임계값 `RAG_MIN_SCORE=0.50` · `RAG_MIN_MARGIN=0.03`, 10문서 인덱스(`index_sha256=bfdc27ec…`).

지표	Before (keyword)	After (RAG)	차이
40문항 분류 정확도	13/40 = 0.3250	34/40 = 0.8500	+0.5250
macro-F1	0.2836	0.8521	+0.5685
E2E 5문항 (실서버 :8100)	1/5 = 0.2000	5/5 = 1.0000	+0.8000
회귀 질의 10문항 (PDF 9쪽)	3/10	10/10	+0.7000
분류 지연 p50	0.0ms	32.43ms	+32.43ms
E2E 요청 지연 p50	750.76ms	819.73ms	+68.97ms
비용 (USD, 로컬 백엔드)	$0.000000	$0.000000	0
retry 횟수	1	1	0
조사 커버리지 (unknown 비율)	18.75%	0.00%	–18.75%p
keyword 폴백률	100.00%	15.00%	–85.00%p



원본은 `artifacts/runs/` 에 있고, 인용한 6개 실행의 요약본은 `doc/reports/evidence/metrics/` , 주장↔증거↔재현 명령 대조표는 `doc/reports/evidence/index.md` 다. 문서에 적힌 숫자가 원본과 어긋나지 않는지 `eval/check_reported_numbers.py` 로 자동 대조한다 (15개 주장 15/13 — 현재 15/15 일치).

유형별로 보면 새로 넣은 4개 유형의 개선이 가장 컸다. F1 기준으로 queue-backlog 0.000 → 1.000, cert-expiry 0.000 → 1.000, connection-pool 0.000 → 1.000, replication-lag 0.000 → 0.857이다.

3.1 결과에서 읽은 것 (한계 포함)

- **데이터 확장(6 → 12시스템)은 분류 정확도를 올리지 못했다.** base 34/40 ≡ extended 34/40으로 문항 단위로 동일했다. 대신 40문항이 조회한 모니터링 80회 중 데이터가 없어 못 본 15회가 0회가 됐다(unknown 18.75% → 0%). 라벨 문항이 질의에 증상을 그대로 적어 둔 탓에 RAG가 질의만 읽어도 대부분 맞았고, 시스템 상태·알림은 중복 신호가 됐다. 그래서 보고서에 "데이터 확장으로 정확도가 올랐다"고 쓰지 않았다.
- **margin 게이트는 2문항을 아꼈다.** 임계값을 0으로 풀면 같은 40문항에서 36/40이지만, 그 2문항은 1·2위 유사도 차이가 0.0155·0.0000인 사실상 동점이고, 1단계에서 정답이 뒤집힌 사례도 정확히 이런 동점이었다. 과적합을 피하려고 0.03을 유지하고 비용을 그대로 적었다.
- **RAG의 비용은 임베딩 1회 호출이다.** 분류 1건당 +32.43ms, 실서버 요청 전체로는 +68.97ms로 요청 시간의 약 9% 수준이었다. 로컬이라 비용은 $0이고, 실 API에서는 이 임베딩 비용이 붙는다.
- **keyword 경로에 남은 결함이 하나 있다.** 회귀 질의 #4(gateway 타임아웃 + payments CPU)는 keyword 경로에서 `cpu-spike` 가 된다. 규칙 순서 문제(B07와 같은 원인)이고, RAG를 켜면 `network-timeout` 으로 고쳐진다. 고치지 않은 게 아니라 RAG로 개선된 지점이다.
- **40문항 라벨을 내가 직접 만들었다.** 자기 평가가 된다. 그래서 질의 의존도가 높은 20문항 (증상·데이터 불일치, 시스템명 없음)을 따로 두고 카테고리별 정확자를 함께 냈다. 전체 수치를 일반화 성능으로 쓰지 않는다.

---

4. 느낀점과 이후에 반영하고 싶은 것

4.1 느낀 점

수업에서 RAG를 배울 때는 "벡터 DB에 넣고 검색하면 된다"는 설명을 들었다. 실제로 해 보니까 그보다 훨씬 정의가 중요했다. **뭐를 입력으로 넣을지**(질의문·시스템·상태·알림), **언제 RAG를 믿고 언제 안 믿을지**(score·margin 하한), **근거를 어떻게 남길지** (metadata·SSE·manifest) 이 세 가지가 사실 구현의 전부였다. 특히 "RAG가 확신하지 못하면 예전 경로로 내려간다"는 게이트 하나를 넣은 뒤에야 기존 동작이 깨지지 않는다는 걸 확신할 수 있었다. 지표도 마찬가지다. 개선을 말하기 전에 기준선을 먼저 재서, 내 개선이 정말 개선인지 숫자로 비교할 수 있는 상태를 먼저 만든 게 이번 과제에서 가장 값졌던 부분이었다.

한 가지 아쉬운 점은, 마지막에야 PDF 9쪽의 예시 질의를 실제로 돌려 봤다는 것이다. 그때 "내 테스트 질의가 자기 시스템에 안 들어간다"는 게 그대로 드러났다. 처음부터 매뉴얼의 테스트 방법을 그대로 따라갔어야 했다는 걸 배웠다.

4.2 이후에 반영하고 싶은 것

1. **실제 장애 티켓·운영 로그로 라벨 세트 재구축** — 지금 라벨은 내가 만든 자기 평가다.
2. **LLM-as-judge 평가 추가** — 지금은 정답 유형만 채점한다. "수집한 수치를 정확히 인용했는가"를 LLM으로 채점하는 평가자를 넣으면 답변 품질까지 볼 수 있다.
1. **Research 서브그래프 분리** — 단일 `research_node`를 로그 분석·메트릭 분석·런북 분석 세 개의 서브에이전트로 쪼개면 멀티에이전트 구조의 이점을 더 크게 볼 수 있다.
1. **Human-in-the-loop 승인 노드** — LangGraph `interrupt()`로 위험한 조치(서비스 재시작 제안) 전에 사람 승인을 받는 노드. 자동 응답의 위험을 낮추는 쪽이다.
1. **메모리 Context 압축** — 대화가 10턴 이상 쌓이면 과거 턴을 요약해 토큰 비용을 줄이는 쪽. 지금은 세션 DB에 그대로 쌓인다.
1. **실 OpenAI 백엔드로 비용·지연 재측정** — 지금의 $0은 로컬 백엔드라서 그렇다. 동시 요청 부하에서의 지연도 재야 한다.

---

5. 제출물

구분	위치
코드	https://github.com/pigpgw/skala-ai-service-mini (`main` 브랜치)
RAG 부품	`agent/rag/{embedder,index,classifier}.py`
그래프 연결	`agent/graph.py::_select_incident_type()` , `replan_check_node`
라우팅 기준	`agent/fake_llm.py::route`
데이터 프로파일	`agent/data_profile.py` , `data/monitoring_extended.json`
평가 하네스	`eval/run_eval.py` , `eval/metrics.py` , `eval/check_reported_numbers.py` , `eval/run_regression_queries.py`
테스트	`tests/unit/` , `tests/integration/` (65개)
측정 원본	`artifacts/runs/<run_id>/` (재생성 가능, 커밋 대상 아님)
근거 대조표	`doc/reports/evidence/index.md` , `before_after.md` , `metrics/`
설계 문서	`doc/rag_design.md` , `doc/evaluation-checklist.md` , `doc/artifact_management_design.md`
아키텍처 다이어그램	`doc/design/architecture.html`
발표자료	`doc/reports/presentation.html` (+PDF·PPTX), `presentation_outline.md`
스프레드시트 입력본문	`doc/reports/spreadsheet_input.md`
리허설 기록	`doc/reports/rehearsal_log.md`
산출물 색인	`doc/reports/index.md`